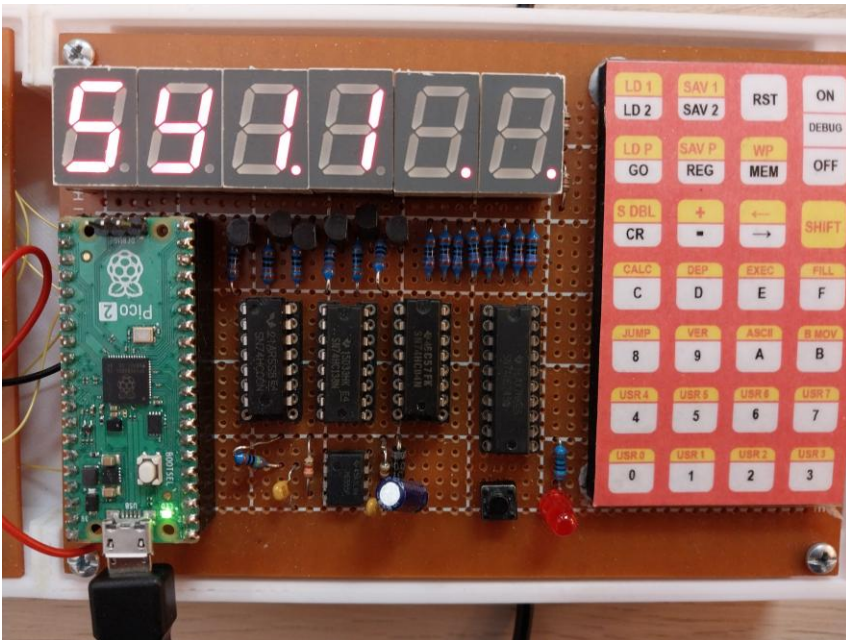




PISYM COMMAND LINE

ABSTRACT

In addition to replication the functionality of the Synertek SYM-1 computer, PiSYM also offers a command line to control configuration, file and tape management for the simulated cassette recorder, and a powerful 6502 debugger.

Brian Campbell

Table of Contents

1	What is PiSYM?	2
2	Accessing the Command Line	2
3	Basic Functions	3
4	Configuration.....	3
4.1	CRT and TTY Circuit	3
4.2	Tape and File Simulation.....	4
4.3	Write Protection	5
4.4	Default Trace Value	5
5	Tape and File Management.....	5
5.1	Under the Hood.....	6
5.2	Path and File Syntax	7
5.3	Saving and Loading Files.....	7
5.4	Device Selection and Management.....	8
5.5	Tape Management.....	8
5.6	File Management.....	9
6	Debugger	10
6.1	General Capabilities.....	10
6.2	Execution Modes.....	10
6.3	Debugger Auto Deactivation	11
6.4	Running and Single Stepping	12
6.5	Program State	13
6.6	Setting Register Values	13
6.7	Dumping Memory.....	14
6.8	Breakpoints	15
6.9	Watches	15
6.10	Trace.....	16
6.11	Single Step Mode	16

1 What is PiSYM?

The PiSYM is a Raspberry Pi Pico 2 based emulator for the Synertek Systems SYM-1 computer of the 1970s. PiSYM emulates a SYM-1 with the full 32K of RAM with both the BASIC and RAE ROMs installed.

Importantly, PiSYM runs at the same speed as a real SYM-1 down to the instruction level. A two-cycle instruction on a 1MHz 6502 will execute in 2 μ s. That same instruction on a PiSYM will also execute in 2 μ s. This is essential as many time-critical functions in the SYM-1 are based on timing loops.

Address aliasing is also replicated in PiSYM. Supermon occupies a 4K block beginning at \$8000 but due to the wiring of the address bus, is aliased starting at \$F000 so the interrupt vectors are at the correct location. Similarly, the 128-byte system RAM that occupies \$A600 to \$A67F is aliased at \$A680 to \$A6FF, \$A700 to \$A77F, and \$A780 to \$A7FF. If you change a value at address \$A600, it will appear at \$A680, \$A700, and \$A780 as well.

The goal of the PiSYM is that it should be impossible to differentiate between it and a real SYM-1 from within the environment for the given hardware installed. In addition to the RAM and ROM, PiSYM also emulates the SY6532 RIOT chip that provides system RAM, I/O, and a Timer. Even 6532 registers are emulated with the DDR and data registers operating the Raspberry Pi Pico 2 GPIO pins.

PiSYM runs Supermon V1.1, BASIC, and RAE unmodified but also has the ability to interwork with more modern peripherals. While PiSYM retains the SYM-1's capability to communicate with a CRT via RS232 or TTY via a 20mA loop, it can also communicate with a terminal via USB. Although PiSYM can save and load data via a cassette recorder, it also has the ability to store and retrieve programs from the internal flash storage of Raspberry Pi Pico 2 or an external SD card. Like the USB port, PiSYM does this transparently to the SYM-1 software such that it thinks it is saving and loading from cassette tape. This even translates to the LOAD and SAVE commands in BASIC.

PiSYM also contains its own debugger. While the SYM-1 has its own debugger and this is emulated along with everything else, this debugger is primitive by modern day standards. The PiSYM debugger handles more advanced features such as breakpoints and watches and unlike the SYM-1 debugger, is capable of debugging Supermon itself.

In addition to emulating a SYM-1 computer, PiSYM offers a command line environment via the USB port to configure and manage it. The command line also offers a more modern debugging capability for 6502 programs that goes beyond the old SYM-1 DEBUG facility.

2 Accessing the Command Line

There are two ways of accessing the command line. Both require USB connection to a terminal program such as PuTTY.

If the DEBUG state of the SYM-1 is on when the Raspberry Pi Pico 2 resets (NOT the SYM-1 RST button), once you open a terminal window to PiSYM, press a key and PiSYM will respond with *Debugger Ready* and display the first instruction of Supermon's RESET routine. You will be in the PiSYM command line environment, and the SYM-1 emulation has not yet begun.

At any time while communicating with the SYM-1 emulation, press the F1 key on your keyboard to break out to the command line.

Type the EXECUTE command to return control to the SYM-1 emulation.

3 Basic Functions

The command line handles the following three basic groups of operations:-

- Configuration
- 'Tape' and file management
- Debugging

The configuration function is straight forward. There are some options that can be set and saved that control the basics of how PiSYM operates.

While the original SYM-1 used cassette tape to save programs, this may be impractical now. As an alternative, PiSYM can simulate cassette-based storage using its internal flash memory or an external SD card. There are commands that assist with managing this medium.

The DEBUG function of the SYM-1 allows single-stepping of a program and that is all. The PiSYM debugger offers many more traditional operations including single stepping such as watches and breakpoints. The debugger can be programmed to automatically switch itself on and off, automatically skipping debugging of sections of code such as all of Supermon.

4 Configuration

There are a small set of configuration options that can be stored in flash memory and automatically loaded when PiSYM starts up.

The general form of the configuration command is:-

```
CONFIG <parameter> <value>
```

You can view the current configuration with the command: -

```
CONFIG LIST
```

You can save the configuration to flash with the command: -

```
CONFIG SAVE
```

You can reload the configuration from flash with the command: -

```
CONFIG LOAD
```

You can erase the configuration from flash and restore the default settings with the command: -

```
CONFIG ERASE
```

The initial state of the configuration (and after CONFIG ERASE) is such that a bare Raspberry Pi Pico 2 with PiSYM installed will operate as a SYM-1 with no external circuitry needed. Simply install PiSYM, connect a terminal program to the USB port, and you have a functioning SYM-1 emulator.

4.1 CRT and TTY Circuit

The hardware for the CRT and TTY interfaces inherently pulls the relevant I/O pins to an inactive state. Supermon looks for activity as it boots in order to calculate the Baud rate. With no terminal, it simply locks up at this point. There are two configuration settings that tell PiSYM to ignore these pins when I/O will always be via USB.

Note that these parameters only affect the input. Output will always use these ports as per normal SYM-1 operation even in parallel with USB.

PiSYM Command Line

Each interface can be activated or deactivated independently. If deactivated, the interface will be ignored. If activated, PiSYM will look to the CRT or TTY port assuming the hardware is in place.

To activate a port, use the commands: -

```
CONFIG CRT ON
CONFIG TTY ON
```

To deactivate a port, use the commands: -

```
CONFIG CRT OFF
CONFIG TTY OFF
```

By default, both ports are deactivated.

4.2 Tape and File Simulation

PiSYM can use the internal flash or an external SD card to simulate a cassette recorder and tapes. The standard SYM-1 software is unaware that it is not communicating with a real cassette deck.

With both simulations switched off, PiSYM will use the actual cassette recorder interface.

To activate the internal flash memory simulation, use the command: -

```
CONFIG FLASH ON
```

To activate the external SD card simulation, use the command: -

```
CONFIG SD ON
```

It is not possible to have both active simultaneously, so turning one on automatically turns the other off.

You can deactivate the simulation with the commands: -

```
CONFIG FLASH OFF
CONFIG SD OFF
```

It is possible to have both deactivated in which case, the cassette interface is used.

If the case of the FLASH and SD parameters, the CONFIG keyword can be omitted.

```
FLASH ON
FLASH OFF
SD ON
SD OFF
```

By default, FLASH simulation is ON and SD card simulation is OFF.

The SYM-1 Resident Assembler Editor (RAE) is designed to use two tape drives with one for reading and the other for writing. This allows it to effectively replace files as they are edited, similar to current storage media. PiSYM goes to great lengths to simulate the operation of a cassette recorder. It cannot operate as two cassette recorders so for the purpose of supporting RAE, can restore the ability to update files. This is controlled by the RAEOVERWRITE parameter.

```
CONFIG RAE OVERWRITE ON
```

will cause PiSYM to overwrite existing RAE file IDs with updated versions.

```
CONFIG RAE OVERWRITE OFF
```

will cause PiSYM to act like a cassette deck, appending updated files to the end of the 'tape'.

4.3 Write Protection

This parameter is not linked to PiSYM hardware but allows you to simulate the behaviour of the SYM-1 hardware.

The SYM-1 used some pins on an I/O port to write protect the RAM in the SY6532 RIOT chip as well as the second to fourth 1K block of on-board RAM. Access to the SY6532 RAM is controlled by calls to the Supermon functions ACCESS and NACCES. Access to the on-board RAM is controlled from Supermon's WP command. Of course, they can also be controlled by directly writing to the I/O port however in the case of the SY6532 RAM, Supermon will itself change the write protection.

With the write protection configuration parameter off, PiSYM will not enforce any write protection. Switching write protection on with this parameter switched off will still allow writing to the relevant memory.

With the write protection configuration parameter on, PiSYM will emulate write protection, disallowing writing to memory locations SYM-1 has deemed write protected. It is worth noting that when SYM-1 starts up, the on-board RAM is write protected by default.

To use the write protection simulation, use the command: -

```
CONFIG WP ON
```

To disable the write protection simulation and make all memory writable, use the command: -

```
CONFIG WP OFF
```

Write protection is OFF by default.

4.4 Default Trace Value

Within the PiSYM debugger is a trace function. As it runs a program, it logs every instruction up to the last thousand. This means that when the debugger encounters a breakpoint, up to 1,000 instructions that led up to that breakpoint can be viewed.

Printing all 1,000 instructions is probably not desirable in all cases. For this reason, the number of instructions to be display can be specified. A default number of instructions can be set in the configuration. Not that this controls the default number displayed, not stored. Even if only 10 instructions are displayed by default, it can be overridden on a case-by-case basis to display all 1,000 if needed.

To set the default, use the command: -

```
CONFIG TRACE <number from 1 to 1000>
```

The default trace is 100 by default.

5 Tape and File Management

As cassette recorders and tapes become harder to come by these days, more modern storage devices can be used to simulate the behaviour of the old media. PiSYM uses either the internal flash memory of the Raspberry Pi Pico 2 or an external SD card. To the SYM-1 emulator, they appear to be ordinary cassette tapes even though they are not.

When using a cassette recorder, files would be stored sequentially. There was no problem with files having the same file ID (equivalent of a filename) on the same cassette tape. Cassette tapes could be swapped in and out as needed. Tapes could be wound forwards and backwards to locate a specific file even if it had the same file ID as another one on the same tape.

All of this is radically different to the way modern file systems work today. To make matters worse, RAE was designed to run with two cassette recorders, allowed programs to be broken up into multiple sequential files (not modular like today's programs), and relied on duplicating file IDs on adjacent files.

PiSYM simulates the operation of a cassette recorder to support all these functions. Unlike a cassette, if you write a file while in the middle of the tape, PiSYM will always append it. It does not corrupt the rest of the 'tape' as a real cassette would.

The internal flash storage has a capacity of about 1MB. The capacity of an SD card is dependent on the card size, limited by the FAT32 formatting.

An SD card managed by PiSYM is readable by a computer capable of reading FAT32 formatted SD cards.

5.1 Under the Hood

To better understand how to operate the tape and file functions, understanding what is happening behind the scenes will be helpful.

In essence, PiSYM must use a conventional file system to behave like a sequential file system on multiple tapes.

The most obvious first step is to use directories to represent tapes. The root tape is simply the root directory. Tape names must at least follow conventional folder naming rules. Although directories can normally have subdirectories, all tapes are in the root directory.

File naming is a little more complicated. Unlike modern file systems, cassette based systems were sequential and did not have issues with duplicated file IDs. PiSYM manages this by using the sequence number as a filename and the file ID as the extension. The sequence number reflects the order of files on a tape even though on tape, there is no literal sequence number. Its existence in PiSYM merely determines the order in which files are served up to the calling function wanting to read from cassette tape.

As an example, assume directory SAMPLE contains the following files: -

```
0001.07
0002.07
0005.01
0010.05
0020.04
0034.07
0100.EE
0110.EE
```

Note that sequence numbers do not have to be contiguous. Files will be presented in the order given even though file IDs are duplicated and out of order.

PiSYM offers renumbering tools to manage sequence numbers to effectively rearrange the order of files on a tape.

Finally, a tape has the concept of a position. This is a sequence number representing where the tape is positioned. The position does not have to reflect the sequence number of an actual file. If the position is set to a value of 30, a read of the next file would be 0034.07. If the position is set to 4 and Supermon is asked to load file ID 07, again file 0034.07 would be read.

5.2 Path and File Syntax

PiSYM can access both internal flash and an external SD card although not simultaneously. That said, it is possible to switch from one to the other. For this reason, it is important to see the actual device being used at any given time.

Simulated cassette tapes can be created and loaded. When displayed, the information is presented as: -

```
<device>:/<tape>
```

For example: -

```
flash:/Games
```

There is a default 'root' tape that is just designated as tape "/.

For example: -

```
sd:/
```

When a tape's contents are listed, each file has not only a file ID but has a sequence number. A current position number is also displayed, representing where the tape is wound to on the cassette deck. The output would be something like: -

```
Files for tape flash:/Games
0001.07
0002.07
0005.01
0010.05
0020.04
>0034.07
0100.EE
0110.EE
```

The > symbol marks the next file to be loaded based on the file position value. As stated earlier, the position value in this example may not necessarily be 34 but can be a value between 21 and 34 inclusive.

5.3 Saving and Loading Files

The commands for assigning sequence numbers and positioning a tape pointer simulate a cassette recorder and fast forwarding and rewinding a tape.

When it comes to saving a file, PiSYM operates a little differently. On a cassette recorder, saving a file in the middle of a tape already containing files will result in corrupting not only the overwritten file but potentially files after. There is no good reason to simulate this sort of data corruption. Instead, PiSYM will always append written files to the end of the tape using the next available sequence number. The necessary tools are available to reposition the file on the tape.

The exception to this is for recognised RAE files. Source files for RAE can easily exceed the available memory, so RAE allows them to be broken down and spread across multiple files. RAE was designed to use two tape recorders, copying data from the source to destination recorders as it goes. In order to simulate the intent behind RAE's design, PiSYM can recognise RAE attempting to write files and can overwrite old files with new ones based on the file ID. This can be activated or deactivated via a configuration parameter.

5.4 Device Selection and Management

The commands: -

```
CONFIG FLASH ON
CONFIG SD ON
```

Activates the internal flash or external SD card interface.

As stated earlier, the CONFIG keyword can be dropped.

Before use, the device must be formatted. Rather than rely on a default device making a mistake very easy, the specific device must be specified.

```
FLASH FORMAT
```

will format the internal flash storage, wiping all files and tapes from the device.

```
SD FORMAT
```

formats the SD card inserted in the external SD card reader.

In both cases, a warning is given that all data on the device will be lost.

Formatting the flash storage is almost instantaneous. Formatting the SD card may take some time and a warning is issued as a result.

There is one command that does access both devices simultaneously. It is intended for those who start using the internal flash storage system and later migrate to the SD card. The COPYTO command copies the content of one device to the other. The command does an entire device copy (not individual tapes or files), starting by formatting the target device.

To copy the internal flash to an external SD card, use the command: -

```
COPYTO SD
```

To copy from an SD card to internal flash, use the command: -

```
COPYTO FLASH
```

Ensure there is less than 1MB of files on the SD card. If the flash storage fills, not all files will be copied.

5.5 Tape Management

Regardless of which device is being used, multiple tapes can be simulated.

Every device has a *root* tape. This tape has no name. It is always there and cannot be deleted or renamed.

Tape names are not case sensitive. "Games" is the same as "games" and "GAMES".

Command	Description
TAPE CREATE <name>	Create a tape with the given name. The tape must not already exist. The name can be any combination of 0-9, a-z, and _ up to 255 characters in length.
TAPE DELETE <name>	Deletes a tape with the given name.
TAPE RENAME <old name> <new name>	Deletes the tape with the name <old name> to <new name>.
TAPE LOAD <name>	Loads the tape with the given name. This simulates inserting a cassette tape into a deck. Files from the tape are accessible from the SYM-1 emulator.
TAPE ?	Display the tape currently loaded
TAPE LIST	List all tapes in the device

Command	Description
TAPE LIST ALL	List all tapes including files in the device
TAPE RENUM[BER] <start> <step>	Renumber all sequence numbers to a new series beginning with <start> and in increments of <step>.
TAPE RENUM[BER] <start and step>	Renumber all sequence numbers to a new series beginning with <start and step> and in increments of <start and step>.
TAPE NEXT <sequence nr>	Forward or rewind such that the next file is equal to or immediately following <sequence nr>.
TAPE REWIND	Rewind the tape to the start.

When dealing with RAE, it is best to create a tape for just RAE files and not to mix other file types. This is due to RAE's method of storing information on tape and would apply equally with actual cassette tapes.

5.6 File Management

The file management commands relate to the currently loaded tape.

Command	Description
FILE LIST	List the files in the tape. The output shows the tape name followed by a list of files with three columns. The first column is the sequence number. The second is the file ID. The last column is the file size in bytes. A ">" symbol indicates the tape position.
FILE DELETE <file ID>	Delete all instances of files with ID <file ID> on the currently loaded tape.
FILE DELETE S<sequence nr>	Delete the file with sequence number <sequence nr> on the currently loaded tape. See Note 1.
FILE RENAME <old file ID> <new file ID>	Renames all instances of files with ID <old file ID> to <new file ID> on the currently loaded tape. See Note 2.
FILE RENAME <sequence nr> <new file ID>	Renames the file with sequence nr <sequence nr> to <new file ID> on the currently loaded tape. See Note 2.
FILE POS[ITION] <old seq nr> <new seq nr>	Repositions file with sequence number <old seq nr> to <new seq nr> on the currently loaded tape. See Note 3.
FILE TYPE <file ID>	Types all files with file ID <file ID> on the currently loaded tape as a hex dump.
FILE TYPE <file ID> B	Types all files with file ID <file ID> on the currently loaded tape interpreting the content as a BASIC program.
FILE TYPE <file ID> R	Types all files with file ID <file ID> on the currently loaded tape interpreting the content as a RAE file.
FILE COPY <file ID> <tape name>	Copy all files with a file ID of <file ID> from the currently loaded tape to the tape named <tape name>. The new files will be appended to the destination tape and will not overwrite any existing files. The destination tape name can be the currently loaded tape.

Note 1: RAE produces files in pairs. It is important not to delete just one RAE file as it will not only make the other file unreadable but will potentially render all remaining files unreadable.

Note 2: RAE stores file IDs inside the first of its two files. Renaming the file using this tool cannot impact the internal name. It is recommended that the best way to change a RAE file ID is to load it and save it from within RAE itself.

Note3: Take care when repositioning RAE files to ensure that file pairs are kept together and stored in the correct sequence with the 9 byte long file first.

6 Debugger

6.1 General Capabilities

The PiSYM debugger allows 6502 assembly language programs to be debugged using modern debugging tools.

The native SYM-1 debugger is still available and can be operated from a CRT, TTY, or keypad and display as per the original SYM-1 as well as via a terminal program communicating over USB. This tool simply allowed programs to be single stepped with access to the memory and registers available in-between.

The PiSYM debugger allows single stepping as well. It also provides watches, breakpoints, and an instruction logging function.

When entering values, they will be interpreted as decimal or hexadecimal based on the context. Memory addresses will always be assumed to be in hexadecimal. DO NOT use \$ or 0x to indicate hexadecimal as it will be assumed.

The CLS command clears the screen. If console logging is in use in the terminal program, this will not necessarily erase the terminal log.

6.2 Execution Modes

When under control of the debugger, there are two modes in which code can be executed: -

- Debugging
- Normal

In the debugging mode, PiSYM is examining each instruction for a watch or breakpoint trigger. Before each instruction is executed, the register state and affected memory address content is logged. Understandably, this slows execution below real time. The 6502 code is not affected as all of these functions take place within the 6502-emulation engine.

In the normal mode, all of these debugging tasks are disabled and the 6502 code runs in real time. This is not just useful for its speed, but in many cases, there will be situations where logging is undesirable such as in timing loops. This allows code of interest to be debugged while ignoring functions of little interest.

In order to assist this process, switching between modes can be done automatically meaning that you can nominate program memory addresses where debugging will be automatically disabled.

Before each instruction is executed in the debugging mode, a snapshot of the registers is taken and stored in a *trace list*. If the instruction references a memory address, that address and its contents are also stored. This snapshot is stored in a list of up to 1,000 instructions with old snapshots being deleted to make way for new ones. This means that the debugger is able to show up to the last 1,000 instructions that led to a breakpoint in the code. In setting a breakpoint, the debugger not only pauses execution, but is able to show the last 1,000 instructions that led to that point.

Since debugging can be programmed to switch on and off based on program addresses, while debugging is off, no snapshots are taken. This means that the trace list will not be filled with

those instructions. This is useful for eliminating calls to Supermon functions for example. The 1,000 places in the trace list will not be occupied with potentially irrelevant data.

Breakpoints nominate program addresses where execution is paused when that instruction is about to be executed. On encountering a breakpoint, PiSYM will pause and enter its single step mode. The program can be single stepped or running under the debugger resumed. It is possible to exit the single step mode to access the full suite of the PiSYM command line instructions. It is also possible to re-enter the single step mode.

There is an option for the breakpoint to simply display the fact that it has been encountered and continue running without pausing.

Watches are used against data memory addresses. The debugger can report when a watch address is encountered for read, writing, or both. When encountered, a line is output noting the address and the data read or written.

As an option, a watch can be programmed to act as a breakpoint as well, providing access to the trace information and single stepping functionality.

When single stepping, simple single character commands are used. Not all debugger command line instructions are available, but exiting and re-entering single stepping is available to provide access to those instructions if needed. The purpose of single stepping mode is to provide single character alternatives for commonly used instructions.

Built into the debugger is the symbol table for Supermon. In any command, a Supermon label can be used as an address. When reporting information, the debugger uses Supermon labels where appropriate. Even if you enter an address as a breakpoint for example, when listing breakpoints, the debugger prints labels where appropriate.

Finally, the debugger allows you to display the current register state of the program, change register values, and print memory content.

6.3 Debugger Auto Deactivation

There are two commands that cause a program to run: -

- RUN : Run the program under control of the debugger
- EXECUTE : Execute the program normally at full speed

The EXECUTE command can be shortened to EXEC.

When running normally under the EXECUTE command, all debugging facilities are disabled.

When running in the debug mode under the RUN command, the debugger's capabilities are available at every instruction. This is not always ideal. For this reason, it is possible to deactivate the debugger automatically based on program addresses using the DEBUG command.

Command	Description
DEBUG OFF <start addr> <end addr>	Disable the debugger for instructions from <start addr> to <end addr>
DEBUG OFF SUPERMON	Disable the debugger for instructions in Supermon from \$8000 to \$8FFF
DEBUG ON <start addr> <end addr>	Enable the debugger for instructions from <start addr> to <end addr>
DEBUG ON SUPERMON	Enable the debugger for instructions in Supermon from \$8000 to \$8FFF

Command	Description
DEBUG LIST	Display the address ranges where the debugger is disabled
DEBUG SAVE	Save debugger addresses to flash memory
DEBUG LOAD	Load debugger addresses from flash memory

By default, DEBUG is ON at every address although if saved to flash memory, debugger information is automatically loaded when PiSYM is reset.

Start and end addresses can be Supermon labels. For example, the command: -

```
DEBUG OFF SAVER 81B7
```

will disable debugging for the SAVER function. One command can cover partial or multiple functions, such as :-

```
DEBUG OFF SAVER 81CA
```

will disable debugging for the SAVER, RESXAF, RESXF, and RESALL functions.

6.4 Running and Single Stepping

As previously discussed, the RUN command will run a program under control of the debugger.

The STEP command will enter the single step mode. The program will still have all of the debugger functions available, but the program can be executed one instruction at a time. When in single step mode, a subset of commands is available as a single character with no enter key required. This makes stepping through code easier. If the full command set is required, it is possible to break out and re-enter single step mode. Single step mode will be detailed below.

It is also possible to specify how many instructions to run before stopping using the command: -

```
RUN <nr instructions>
```

After the specified number of instructions, control is returned to the command line.

6.5 Program State

The general form of the program state is: -

<address> <code> <instruction> <PC> <A> <X> <Y> <SP> <SR> [<memory>]

Where: -

<address>	Address of next instruction
<code>	Object code for the next instruction
<instruction>	Instruction text
<PC>	PC = <hex value> : Program Counter value
<A>	A = <hex value> : Accumulator value
<X>	X = <hex value> : X Register value
<Y>	Y = <hex value> : Y Register value
<SP>	SP = 01<hex value> : Stack Pointer value
<SR>	SR = <Status register bits> N/n : Negative Set/Clear V/v : Overflow Set/Clear 1 B/b : Break Set/Clear D/d : Decimal Set/Clear I/i : Interrupt Mask Set/Clear Z/z : Zero Set/Clear C/c : Carry Set/Clear
<memory>	Addr <hex value> = <hex value> If a memory address is referenced, the address and content prior to the instruction being executed is given.

The instruction text will contain the mnemonic and parameter in the standard 6502 format. If the parameter is an address contained within the symbol table, the symbol will be used with the address in parentheses after.

When displaying a memory address and its content, the debugger will calculate the effective address, even if indexed indirect or indirect indexed addressing is used.

The current state can be displayed in the debugger using the command: -

REG

or: -

?

6.6 Setting Register Values

The debugger offers a simple way to set the value of one of the registers.

Command	Description
PC = <addr>	Set the program counter to <addr>
SP = <addr>	Set the stack pointer to <addr>. The 01 prefix is assumed so only the single byte value is set.
A = <value>	Set the accumulator to <value>
ASR = <value>	Set the accumulator to <value> and set the N and Z bits of the status register accordingly.

Command	Description
X = <value>	Set the X index register to <value>
XSR = <value>	Set the X index register to <value> and set the N and Z bits of the status register accordingly.
Y = <value>	Set the Y index register to <value>
YSR = <value>	Set the Y index register to <value> and set the N and Z bits of the status register accordingly.
SR = <flag> [<flag>...]	Set or clear the status register flags. Flag values are: - N - Negative V - Overflow D - Decimal I - Interrupt mask Z - Zero C - Carry Specifying the flag causes it to be set. Prefixing it with ! causes it to be cleared. For example: - SR = !Z C !N D clears Z and N and sets C and D. Other flags remain unaltered.

6.7 Dumping Memory

The DUMP command allows you to print out a range of memory. Use the command: -

```
DUMP <start address> <number of bytes>
```

The start address and number of bytes are both hexadecimal values.

The printout begins at a 16-byte boundary and covers the address range specified. For example, the command:-

```
DUMP EB95 17
```

will produce: -

```
Addr  0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
EB90                86 EE C0 50 B0 02 E6 EE 20 A3 EB
EBA0  4C 5E B0 6C F2 00 20 AC EB 4C 5E B0
```

A print of the stack content can be provided using the command: -

```
STACK
```

This command will print the stack pointer followed by all of the data from there to the top of the stack. For example: -

```
SP: 01F9
01FA: 48
01FB: 24
01FC: 76
01FD: 02
01FE: 01
01FF: 02
```

Bear in mind that the stack pointer always points to the next address to be written so never contains valid data at that address.

6.8 Breakpoints

A breakpoint is a location in your program that when encountered, stops execution.

Command	Description
BREAKPOINT SET <addr> BP SET <addr>	Set a breakpoint at <addr>
BREAKPOINT SET <addr> O BP SET <addr> O	Set a breakpoint at <addr> to output only the fact that the program got to that point. Execution under the debugger automatically continues.
BREAKPOINT CLEAR <addr> BP CLEAR <addr>	Clear the breakpoint at <addr>
BREAKPOINT CLEAR ALL BP CLEAR ALL	Clear all breakpoints
BREAKPOINT LIST BP LIST	List all breakpoints

When a breakpoint is encountered, execution stops and PiSYM enters its single step mode unless the O parameter is specified. Program execution can continue using the single step R command.

The <address> parameter can be a program label from within Supermon. Of course, debugging within Supermon must not be turned off for this to work.

6.9 Watches

A watch is generally attached to a data memory address. A watch can be set to trigger when a program attempts to read, write, or read or write to that address.

Watches are only monitored while a program is in debug mode. If debugging is off, watches are ignored.

When a watch triggers, a message giving the details of the memory access is produced. It is also possible to cause execution to stop and cause PiSYM to enter single step mode, effectively turning the watch into a breakpoint as well.

Command	Description
WATCH READ <addr> WATCH R <addr>	Create a watch to trigger on a read access of <addr>
WATCH READ <addr> BREAK WATCH R <addr> BREAK	Create a watch to trigger on a read access of <addr> and break to single step mode when encountered
WATCH WRITE <addr> WATCH W <addr>	Create a watch to trigger on a write access of <addr>
WATCH WRITE <addr> BREAK WATCH W <addr> BREAK	Create a watch to trigger on a write access of <addr> and break to single step mode when encountered
WATCH READWRITE <addr> WATCH RW <addr>	Create a watch to trigger on a read or write access of <addr>
WATCH READWRITE <addr> BREAK WATCH RW <addr> BREAK	Create a watch to trigger on a read or write access of <addr> and break to single step mode when encountered
WATCH RANGE READ WRITE READWRITE <start addr> <end addr> [BREAK] WATCH RANGE R W RW <start addr> <end addr> [BREAK]	As above but for the memory range from <start addr> to <end addr>

Command	Description
WATCH CLEAR <addr>	Clear watch at <addr>
WATCH RANGE CLEAR <start addr> <end addr>	Clear watches from <start addr> to <end addr>
WATCH LIST	List all watches

For Supermon memory addresses, labels can be used, however the label only refers to a single address. For example, although the variable SCPBUF refers to a block of memory from \$A600 to \$A61F, when used for a watch it only refers to the start address. To add a watch to the whole range, use the command: -

```
WATCH RANGE RW SCPBUF A61F
```

6.10 Trace

As a program executes, up to 1,000 instructions are held in an internal log and can be printed on command. This means that when a breakpoint is encountered whether from a programmed breakpoint or a watch with the BREAK indicator set, a list of up to the last 1,000 instructions can be displayed leading up to that point.

While debugging, every instruction is logged. If execution enters a region where debugging is switched off, no logging takes place. This can be useful for skipping functions that do not matter such as opening and closing write protection access and the bit-by-bit emission of a character to the CRT will all of the delay loops. The 1,000 instructions can be used for things that matter.

Command	Description
TRACE <nr instructions>	Display the trace of the last <nr instructions> instructions
TRACE	Display the trace of the last default number of instructions as defined in the configuration.

6.11 Single Step Mode

When PiSYM enters its single step mode, the prompt changes from the usual dollar (\$) symbol to a colon (:).

A small subset of commands are available as single characters. The ENTER key is not required.

Command	Description
[ESC]	The escape key exits single step mode back to the command line. Type the STEP command to re-enter single step mode.
[Space]	The space key executes the next instruction. Equivalent to 'step into' in many debuggers.
?	Display current program state. Same as REG or ? on the command line.
C	Continue running program under the debugger
R	Run until return. Equivalent to 'run until return' in many debuggers.
S	Display the stack
T	Display the default number of trace instructions.
X	Execute the next instruction or the entire subroutine if it is a JSR instruction. Equivalent to 'step over' in many debuggers.

